ApplyWizz Signal

Complete Product Blueprint v1.0

PRD • TRD • App Flow • UI/UX Design Brief • Backend Schema • Implementation Plan

Every important customer conversation becomes structured company memory automatically.

Internal ApplyWizz planning document • September 2026

Document Map

#	Document	What it locks
01	PRD	Product, users, roles, features, policies, V1/V2 scope, user stories, success metrics
02	TRD	React/Next.js, Supabase, Microsoft Graph, Vexa abstraction, OpenAI, queues, email, CRM, deployment
03	App Flow	Every page, click, redirect and Admin → Manager → Account Manager journey
04	UI/UX Design Brief	Design system, layouts, components, responsive behavior and every important screen
05	Backend Schema	Exact PostgreSQL tables, columns, enums, FKs, indexes, RLS, storage, APIs, events and queues
06	Implementation Plan	Dependency-based build order, tests, milestones and definition of done

This blueprint follows the six-document pre-coding framework supplied for the project. Each document is a source of truth and should be updated explicitly if implementation decisions change.

Locked global decisions

- Product: ApplyWizz Signal. Bot name: ApplyWizz Meeting Assistant.
- Initial deployment: internal ApplyWizz, approximately 10–15 Account Managers.
- Primary platform: Microsoft Teams.
- Frontend: Next.js + React + TypeScript.
- Backend/system of record: Supabase PostgreSQL with RLS.
- Meeting bot: provider abstraction; Vexa is the first adapter candidate, not the product core.
- AI: OpenAI through an internal provider abstraction.
- Eligible meetings are captured automatically unless policy or an approved exception prevents capture.
- External/client-safe output and internal/company intelligence are structurally separated.
- No coding until product, technical architecture, app flow, UX and backend schema are approved.

01 — Product Requirements Document (PRD)

1. Product definition

Field	Decision
Name	ApplyWizz Signal
Tagline	Every customer conversation, understood.
Category	Company-controlled meeting intelligence and organizational memory
Primary users	Admin, Senior Manager, Manager, Account Manager
Initial meeting platform	Microsoft Teams
Primary promise	Schedule the meeting normally. ApplyWizz handles everything else.

2. Problem

ApplyWizz account managers repeatedly conduct onboarding, progress-review, renewal, partnership and follow-up meetings. Important decisions, customer concerns, promises and next steps are easily lost across notes, inboxes and individual memory. CRM updates are inconsistent, managers cannot attend every call, and customer knowledge can disappear when an employee changes role or leaves.

A recorder/transcriber solves only capture. ApplyWizz Signal must turn conversations into operational company intelligence: factual follow-up, internal signals, actions, commitments, CRM history and searchable relationship memory.

3. Product principles

Principle	Requirement
Company controlled	Organization policy controls eligible capture; ordinary employees do not have a casual disable toggle.
Automatic by default	Eligible Teams meetings become bot jobs without manual bot invitation.
Exceptions, not repeated activation	Employee may request a one-meeting do-not-record exception; authorized approver decides.
External/internal separation	Client-safe communication cannot contain internal risk, opportunity, coaching or speculative intelligence.
Hierarchy aware	Access and delivery understand Admin → Senior Manager → Manager → Account Manager.
Observable automation	Meeting lifecycle, failures, retries and external deliveries are visible.
Evidence first	Important AI claims should trace to transcript evidence where practical.
Institutional memory	Customer history belongs to the organization, not one employee.

4. Personas

Role	Primary responsibility	Primary question
Admin	People, policy, integrations, audit, reliability	Are intended meetings captured and processed safely?
Senior Manager	Cross-team patterns and escalations	What is changing across customers and teams?
Manager	Direct-report customer intelligence	What needs my attention without attending every call?
Account Manager	Customer meetings and follow-through	What do I need to do before/after my meetings?

5. Customer lifecycle

```
Payment → Onboarding → ~15-Day Progress Review → Follow-ups / Progress Reviews → Renewal → Expansion / Continued Relationship
```

6. V1 must-have scope

Domain	Capabilities
Organization	Organization, roles, permissions, hierarchy, teams/departments foundation
People	Admin adds work email, role, manager, team, Meeting Intelligence status
Microsoft	Calendar connection, event discovery, webhook handling, subscription renewal, reconciliation
Policy	Eligibility rules, exclusions, reason codes, recording-exception approval
Bot	Provider-neutral bot job, Vexa adapter, join/lobby/failure states, deduplication
Capture	Participant metadata, recording reference where allowed, structured transcript
AI	Facts, external summary, internal intelligence, decisions, actions, commitments, evidence
Email	Review required/window/immediate/disabled modes; safe recipient resolution
Manager	Direct-report visibility, delivery preferences, needs-attention dashboard
CRM	Customer association and idempotent meeting intelligence sync
Memory	Customer timeline, open commitments, decisions, relationship snapshot
Ask Signal	Ask This Meeting and Ask This Customer with authorization
Governance	RLS, audit, retention foundation, visible failures and retries

7. V1.5 / later

- Daily/weekly manager digest and leadership brief.
- Pre-meeting brief using previous summary, open commitments and unresolved concerns.
- Meeting-to-meeting change detection.
- Semantic organization-wide search and Ask My Team/Ask Organization.
- Customer Voice aggregation and recurring objection/feature-request analysis.
- Overdue commitment alerts and richer workflow integrations.
- Custom roles/templates, transcript exports and richer evidence navigation.
- Zoom and Google Meet after Teams reliability is proven.

8. Explicitly out of scope for V1

- Public Otter/Fireflies replacement; public self-service signup or billing.
- Desktop/local recorder, Chrome extension or native mobile app.
- Employee performance scoring or covert surveillance.
- Sales forecasting engine or CRM replacement.
- Live coaching during the meeting.
- Universal public cross-tenant bot guarantee.
- External customer portal and agent marketplace.

9. Key user stories

Role	Story
Admin	Add an employee, assign role/manager and activate Meeting Intelligence.
Admin	Understand why a bot joined/did not join and whether transcript, AI, email and CRM completed.
Admin	Control capture and external sharing so employees cannot casually bypass company policy.

AM	Schedule Teams normally and have the Meeting Assistant appear automatically.
AM	Request a legitimate one-meeting recording exception with a reason.
AM	Receive summary, actions and commitments without manually rewriting the call.
Manager	See risks, opportunities, overdue commitments and flagged conversations across direct reports.
Manager	Choose summary/transcript delivery according to policy.
Leadership	Preserve customer knowledge and recurring signals as company memory.

10. Recording exception workflow

1. Eligible meeting is automatically marked for capture.
2. Account Manager selects Request not to record and enters a required reason.
3. Authorized Admin/Manager receives the request.
4. Approve → bot is cancelled/suppressed for that meeting.
5. Reject → organization capture policy remains active.
6. Unresolved by cutoff → organization-configured default applies.
7. Request, decision, reason and timestamps are audited.

Participant recording/transcription notice or consent remains a separate compliance concern from employee exception policy.

11. External vs internal AI

Client-safe / external	Internal / ApplyWizz only
Meeting overview	Sentiment / relationship signal
Key discussion points	Objections and blockers
Explicit decisions	Renewal/churn/expansion signals
Action items	Stakeholder observations
Next steps	Internal recommendations
Explicit commitments	Escalation and manager-only notes

12. Success metrics

Metric	V1 target / definition
Meeting detection	≥99% of eligible calendar meetings represented in Signal
Bot duplication	0 duplicate active bots for same meeting generation
Bot join success	Pilot target ≥95%; every failure investigated
Transcript completion	≥98% of successfully captured meetings
AI completion	≥98% after transcript availability
External leakage	0 critical internal-intelligence leakage incidents
Email reliability	≥99% accepted/sent jobs excluding invalid addresses
CRM sync	≥99% eventual success with retries
Manual work	No manual start-recorder/invite-bot step in normal flow
Auditability	Every exception and external delivery can answer who/what/why/when

13. Golden path

```
Admin adds employee → role + manager → Microsoft calendar → AM schedules Teams → event detected → policy evaluated → exactly one bot scheduled → bot joins → transcript → external summary + internal intelligence → actions/commitments → safe email → manager visibility → CRM → customer memory
```

02 — Technical Requirements Document (TRD)

1. Architecture

ApplyWizz PostgreSQL is the source of truth. Microsoft, Vexa, OpenAI, email and CRM are replaceable/synchronized providers.

```
Next.js / React
      │
Supabase Auth + PostgreSQL + RLS
      │
      ├─ Microsoft Graph / Calendar
      ├─ Durable Queues + Worker
      │       └─ Meeting Orchestrator
      │               └─ MeetingBotProvider
      │                       └─ Vexa → Microsoft Teams
      ├─ Transcript / Storage
      ├─ AIProvider → OpenAI
      ├─ EmailProvider
      ├─ CRMProvider → ApplyWizz CRM
      └─ Relationship Memory
```

2. Locked stack

Layer	Decision	Purpose
Frontend	Next.js + React + TypeScript	Role-aware web app
UI	Tailwind CSS + accessible primitives + Lucide	Consistent enterprise UI
Backend	Supabase	Auth, PostgreSQL, RLS, Storage, Realtime
Jobs	Durable Postgres-backed queue + worker	Retryable long-running orchestration
Calendar	Microsoft Graph	Calendar/Teams event metadata
Meeting bot	MeetingBotProvider; Vexa first adapter	Join Teams, capture/transcript
AI	AIProvider; OpenAI first adapter	Facts, summaries, intelligence, Ask Signal
Email	EmailProvider abstraction	Transactional delivery
CRM	CRMProvider; ApplyWizz adapter	Customer history/actions
Deployment	Web host + Supabase + worker runtime	Separate web/data/background concerns

3. Architecture rules

- Modular monolith plus asynchronous workers; no premature microservices.
- Provider-specific status/IDs never leak into core product types or UI.
- Webhooks improve freshness; reconciliation protects correctness.
- Every side effect uses an idempotency key.
- Privileged credentials remain server-side.
- Structured JSON is validated before AI output is persisted/published.

4. Logical modules

Module	Responsibility
Identity	Auth, profile, organization membership
Organization	Roles, permissions, hierarchy, teams
Calendar	Connections, subscriptions, event sync/reconciliation
Meetings	Canonical meeting, attendees, lifecycle
Policy	Eligibility, exclusions, reason codes, exceptions
Bot Orchestrator	Bot intent, provider calls, status, retries
Transcript	Fetch/normalize transcript and speakers

AI Intelligence	Facts, external summary, internal intelligence, evidence
Email	Recipient resolution, review window, delivery
CRM	Customer association and sync
Memory	Timeline, commitments, relationship snapshot, retrieval
Audit/Observability	Activity log, metrics, queue/failure visibility

5. Microsoft flow

```
Employee activated → Microsoft connection → calendar subscription → initial sync → event webhook → canonical meeting → policy → bot intent. Periodic reconciliation repairs missed notifications and subscription lifecycle jobs renew expiring subscriptions.
```

6. Policy priority

8. Organization disabled.
9. Employee Meeting Intelligence disabled.
10. Unsupported meeting mechanism.
11. Explicit admin exclusion.
12. Approved recording exemption.
13. Sensitive/private/internal exclusion.
14. Role/team policy.
15. External/client rule.
16. Organization default policy.

7. Meeting bot provider contract

```
MeetingBotProvider
- createBot(meeting)
- cancelBot(botJob)
- getBotStatus(providerBotId)
- getTranscript(providerBotId)
- getRecording(providerBotId)

Normalized states:
scheduled → provisioning → joining → lobby? → active → ending → completed
Terminal: cancelled | failed | removed | timeout
```

8. AI pipeline

```
Meeting completed → transcript → speaker normalization → fact extraction → external-safe summary → internal intelligence → evidence verification → actions/commitments → email → manager workflow → CRM → relationship memory
```

9. Environment variable names

Category	Names
Supabase	NEXT_PUBLIC_SUPABASE_URL; NEXT_PUBLIC_SUPABASE_ANON_KEY; SUPABASE_SERVICE_ROLE_KEY
Microsoft	MICROSOFT_CLIENT_ID; MICROSOFT_CLIENT_SECRET; MICROSOFT_TENANT_ID; MICROSOFT_WEBHOOK_CLIENT_STATE
Vexa	VEXA_BASE_URL; VEXA_API_KEY
OpenAI	OPENAI_API_KEY
Email	EMAIL_PROVIDER_API_KEY; EMAIL_FROM_ADDRESS
CRM	APPLYWIZZ_CRM_BASE_URL; APPLYWIZZ_CRM_API_KEY
App	APP_BASE_URL; WEBHOOK_BASE_URL; ENCRYPTION_KEY; LOG_LEVEL

10. Repository structure

```
applywizz-signal/
├─ apps/web/{app,components,features,tests}
├─ packages/{domain,database,auth,microsoft,meeting-bots,ai,email,crm,observability}
├─ workers/orchestrator
├─ supabase/{migrations,functions,tests,seed.sql}
├─ docs/{product,architecture,runbooks}
└─ tests/e2e
```

11. API surface

Method	Endpoint	Purpose
POST	/api/people	Create person
PATCH	/api/people/:id	Update person/reporting/settings
GET	/api/meetings	Authorized meeting list
GET	/api/meetings/:id	Meeting workspace
POST	/api/meetings/:id/exemption	Request exception
POST	/api/exemptions/:id/approve	Approve exception
POST	/api/exemptions/:id/reject	Reject exception
POST	/api/meetings/:id/reprocess	Reprocess failed stage
POST	/api/meetings/:id/email/send	Send approved external email
POST	/api/ai/meeting/:id/ask	Ask This Meeting
POST	/api/ai/customer/:id/ask	Ask This Customer
POST	/api/webhooks/microsoft/calendar	Calendar notifications
POST	/api/webhooks/microsoft/lifecycle	Subscription lifecycle
POST	/api/webhooks/vexa	Bot events
POST	/api/webhooks/email	Delivery events

12. Durable queues

Queue	Jobs
calendar-events	Refresh/create/update/cancel canonical meetings
meeting-policy	Evaluate/re-evaluate eligibility
bot-provisioning	Create/cancel/retry bot
bot-status	Synchronize provider lifecycle
transcript-processing	Fetch/normalize transcript
ai-processing	Facts, summaries, intelligence, evidence
email-delivery	Review-window expiry and send
crm-sync	Customer/meeting intelligence sync
search-indexing	Refresh retrieval corpus
notifications	User-facing alerts

13. Reliability & security

- Webhook handlers validate and enqueue quickly; heavy work runs asynchronously.
- Calendar reconciliation recovers missed notifications.
- Retries use bounded backoff and dead-letter state.
- Completed is never shown when required downstream stages failed.
- RLS is enabled on every exposed table and tested with allow/deny cases.
- Service-role and provider secrets are server-side only.
- External participants never receive internal workspace access.

- Audit covers policy, role, exception and external-delivery changes.

03 — App Flow

1. Role entry

```
Sign In → membership + role resolution → Admin: /admin/overview | Senior Manager/Manager: /manager/overview | Account Manager: /home
```

2. Page map

Role	Route	Purpose
All	/login	Authentication
Admin	/admin/overview	Meeting operations
Admin	/admin/meetings	All meetings
Admin	/admin/meetings/:id	Meeting detail + technical log
Admin	/admin/people	People
Admin	/admin/people/new	Add person
Admin	/admin/people/:id	Person detail
Admin	/admin/teams	Teams/departments
Admin	/admin/exceptions	Recording exception queue
Admin	/admin/policies	Recording/email/manager/AI/retention
Admin	/admin/integrations	Provider health/configuration
Admin	/admin/audit	Audit log
Manager	/manager/overview	Team intelligence
Manager	/manager/team	Direct reports
Manager	/manager/team/:id	Employee detail
Manager	/manager/meetings	Authorized meetings
Manager	/manager/customers	Authorized customers
Manager	/manager/actions	Team actions/commitments
Manager	/manager/intelligence	Risks/opportunities/patterns
AM	/home	Today's meetings + attention
AM	/meetings	Own meetings
AM	/meetings/:id	Meeting workspace
AM	/customers	Customers
AM	/customers/:id	Relationship Memory
AM	/actions	Actions/commitments
Authorized	/ask	Ask Signal

3. Add Person journey

```
People → Add Person → work email → name → role → manager → department/team → Meeting Intelligence ON → policy → Save → Person Detail → Microsoft status → initial calendar sync
```

4. Normal meeting journey

```
AM schedules Teams → event detected → Upcoming → policy → Meeting Assistant scheduled → joining/lobby/active → meeting ends → transcript processing → AI processing → Completed Meeting → client email → manager workflow → CRM → customer memory
```

5. Exception journey

```
Upcoming Meeting → Request not to record → reason → Submit → Approval pending → Approve: bot suppressed / Reject: bot
```

```
stays scheduled → audit
```

6. Client email journey

```
External summary ready → delivery mode → recipient resolution → external-safe renderer → review/countdown if required → send → delivery state → audit + CRM activity
```

7. Manager journey

```
Overview → Needs Attention → risk / overdue commitment / exception / workflow failure → source meeting/customer → evidence → action
```

8. Customer journey

```
Customers → Customer → relationship snapshot → commitments → meetings → decisions → risks/opportunities → Ask This Customer → grounded answer with source meetings
```

9. Loading, empty and error states

Context	Required state
Calendar	Connecting / syncing / disconnected + reconnect
Bot	Scheduling / joining / lobby / active / failed
Post-meeting	Processing transcript / creating intelligence / delayed
Email	Draft / review / scheduled / sending / sent / failed
CRM	Syncing / synced / failed
Empty Admin	No meetings yet; add people/connect Microsoft
Empty AM	No meetings today; eligible Teams meetings appear automatically
Permission	Restricted by organization policy; no data leakage

10. Redirect logic

- Unauthenticated protected route → /login.
- Inactive membership → access-pending/inactive state.
- Unauthorized entity route → permission-denied/404-safe response.
- After Add Person → Person Detail.
- After exception submit → meeting with pending status.
- After approval/rejection → exception queue with updated meeting state.
- After logout → /login.

04 — UI/UX Design Brief

1. Design goal

A serious enterprise intelligence product: clean, calm, trustworthy and operational. It should not look like a generic AI dashboard or a clone of a meeting recorder.

2. Experience by role

Role	UX character	Primary content
Admin	Operational, moderately dense	Status, failures, policy, people, integrations
Senior Manager	Executive, pattern-first	Cross-team risks/opportunities/trends
Manager	Attention-first	Direct-report signals, commitments, exceptions
Account Manager	Low-friction	Today's meetings, completed meeting, actions

3. Visual system

Token	Direction
Mode	Light primary; dark later
Primary	Deep ApplyWizz blue
Background	White / cool light gray
Text	Near-black navy/charcoal
Success	Green only for healthy/completed
Warning	Amber for waiting/review
Critical	Red for failure/high-risk
Typography	Modern sans; strong hierarchy
Corners	8–12 px
Shadows	Subtle; borders/spacing do most work
Icons	Consistent line-icon set
Density	Tables for operations; cards only when scanning improves

4. Core components

- AppShell, SidebarNav, PageHeader, Breadcrumbs.
- MetricCard, DataTable, StatusBadge, LifecycleStepper.
- MeetingCard, AttentionItem, DetailDrawer.
- TranscriptViewer with speaker/timestamp/evidence anchor.
- ActionItemRow, CommitmentRow, RelationshipTimeline.
- PolicyRuleBuilder and ReviewCountdown.
- Accessible ConfirmModal and full-screen mobile sheets.

5. Screen inventory

#	Screen	Key content
01	Admin Overview	KPIs; Live Now; Needs Attention; Upcoming; System Health
02	Admin Meetings	Filters; table; lifecycle/status columns
03	Admin Meeting Detail	Summary; Transcript; Internal Intelligence; Actions; Attendees; Ask AI; Technical Log
04	People	Search; filters; Add Person; Microsoft/Meeting AI status
05	Add Person	Email; role; manager; team; Meeting

		Intelligence; policy
06	Person Detail	Profile; reporting; calendar; policy; stats; audit
07	Teams	Departments/teams; managers; members
08	Policies	Recording; Exceptions; Email; Manager Access; AI; Retention
09	Exceptions	Pending/Approved/Rejected; reason; decision
10	Integrations	Microsoft; bot; OpenAI; email; CRM health
11	Audit	Actor; action; entity; date; details
12	Manager Overview	KPIs; Needs Attention; risks/opportunities
13	My Team	Employees; meetings; customers; actions; signals
14	Employee Detail	Recent meetings; customers; commitments; signals
15	Manager Intelligence	Risks; opportunities; recurring objections
16	AM Home	Needs Attention; Today's Meetings; recent; commitments
17	Upcoming Meeting	Participants; assistant scheduled; exception action
18	Completed Meeting	Summary; transcript; actions; Ask AI; email status
19	Actions	Open/in progress/completed/overdue
20	Customer Memory	Snapshot; timeline; commitments; decisions; signals; Ask
21	Ask Signal	Scope; grounded answer; sources

6. Required AM simplicity

```
10:00 AM
Bowling Green State University
Progress Review
• ApplyWizz Meeting Assistant scheduled

[View]   [Request not to record]

No Start Recording button.
No Invite Bot button.
No Vexa/provider settings.
```

7. Required Manager wow moment

```
WHAT NEEDS YOUR ATTENTION

🔴 Renewal concern — BGSU
Customer raised a renewal concern yesterday. [View evidence]

🟠 Commitment overdue — Toledo
Proposal promised 8 days ago is still open. [Open commitment]

🟢 Expansion signal — Kent State
Customer expressed interest in expansion. [View meeting]
```

8. Responsive/accessibility

- Desktop full sidebar; tablet collapsible sidebar; mobile compact/bottom navigation for AM.
- Admin tables become scrollable or card/list views on narrow screens.
- Never rely on color alone for status.
- WCAG AA contrast, keyboard navigation, visible focus and labeled forms.
- Production body text should remain comfortably readable; errors appear beside fields.
- Transcript timestamps and speaker labels must remain accessible.

05 — Backend Schema

1. Domain relationship

```
Organization
├─ Memberships — Roles / Permissions — Reporting hierarchy
├─ Teams / Departments
├─ Customers
│  └─ Meetings
│     ├─ Attendees
│     ├─ Policy Decision / Exemption
│     ├─ Bot Jobs / Lifecycle
│     ├─ Recording / Transcript / Segments
│     ├─ AI Runs / External Summary / Internal Intelligence
│     ├─ Decisions / Actions / Commitments
│     ├─ Emails / Recipients
│     └─ CRM Sync
└─ Audit Events
```

2. Enums

Enum	Values
membership_status	invited, setup_required, active, suspended, deactivated
role_key	admin, senior_manager, manager, account_manager
meeting_eligibility	pending, record, exclude, pending_exception, unsupported
meeting_status	upcoming, live, processing, completed, cancelled, failed
bot_status	scheduled, provisioning, joining, lobby, active, ending, completed, cancelled, failed, removed, timeout
exemption_status	requested, approved, rejected, cancelled, expired
transcript_status	pending, processing, completed, failed
ai_run_status	queued, running, completed, failed
email_status	draft, awaiting_review, scheduled, sending, sent, partially_failed, failed, cancelled
action_status	open, in_progress, completed, cancelled, overdue
sync_status	pending, running, succeeded, failed, dead_letter

3. Exact table catalog

Table	Key columns	Indexes / constraints
organizations	id uuid PK; name text; slug text UQ; status text; timezone text; created_at timestamptz; updated_at timestamptz	slug
profiles	id uuid PK→auth.users; display_name text; email citext; avatar_url text?; created_at; updated_at	email
roles	id uuid PK; organization_id uuid?; key text; name text; hierarchy_level int; is_system_role bool	organization_id,key
permissions	key text PK; description text	key
role_permissions	role_id uuid FK; permission_key text FK	role_id,permission_key UQ
departments	id uuid PK; organization_id FK; name text; status text	organization_id,name
teams	id uuid PK; organization_id FK; department_id?; name; manager_membership_id?; status	organization_id,name
organization_memberships	id; organization_id; user_id?; work_email citext; display_name; role_id; department_id?; team_id?; manager_membership_id?; status; meeting_ai_enabled; created_at; updated_at	org,work_email UQ; manager; role

calendar_connections	id; membership_id; provider; provider_user_id; status; scope_metadata jsonb; last_sync_at	membership UQ; provider_user_id
provider_subscriptions	id; calendar_connection_id; external_subscription_id; resource; expires_at; status; last_notification_at; last_renewed_at	external_subscription_id UQ; expires_at
calendar_sync_cursors	id; calendar_connection_id; cursor; window_start; window_end; updated_at	connection UQ
customers	id; organization_id; external_crm_id?; name; domain?; lifecycle_stage; owner_membership_id?; status; metadata jsonb; timestamps	org,name; external_crm_id
customer_contacts	id; customer_id; email?; name?; title?; status	customer,email
meetings	id; organization_id; owner_membership_id; customer_id?; provider; external_event_id; external_meeting_id?; meeting_url?; title; meeting_type?; scheduled_start/end; actual_start/end?; eligibility_status; reason_code?; lifecycle_status; timestamps	org,provider,event UQ; owner,start; customer,start
meeting_attendees	id; meeting_id; email?; display_name?; participant_type; invited; attended?; joined_at?; left_at?; provider_participant_id?	meeting,email; meeting,attended
meeting_policy_sets	id; organization_id; name; is_default; status	one default/org
meeting_policy_rules	id; policy_set_id; priority; rule_type; conditions jsonb; decision; reason_code; enabled	policy,priority
meeting_policy_decisions	id; meeting_id; policy_set_id?; decision; reason_code; rule_id?; evaluated_at; context_snapshot jsonb	meeting,evaluated_at
recording_exemption_requests	id; meeting_id; requested_by; reason; status; reviewed_by?; review_notes?; requested_at; reviewed_at?; expires_at?	meeting,status
meeting_bot_jobs	id; meeting_id; provider; provider_bot_id?; status; generation; scheduled_for; joined_at?; ended_at?; failure_code?; failure_message?; provider_metadata jsonb; timestamps	meeting,generation UQ; provider_bot_id
meeting_lifecycle_events	id; meeting_id; bot_job_id?; event_type; source; payload jsonb; occurred_at	meeting,occurred_at
recordings	id; meeting_id; provider; storage_path?; external_reference?; duration_seconds?; status; retention_expires_at?	meeting,status
transcripts	id; meeting_id; provider; language?; status; version; created_at; completed_at?	meeting,version UQ
speakers	id; transcript_id; provider_speaker_key?; display_name?; attendee_id?	transcript,provider_key
transcript_segments	id; transcript_id; sequence_number; speaker_id?; start_ms; end_ms; text; confidence?	transcript,sequence UQ
ai_runs	id; meeting_id; run_type; model; prompt_version; input_version; status; started_at; completed_at?; error_code?; usage_metadata jsonb	meeting,run_type,status
meeting_external_summaries	id; meeting_id; ai_run_id; version; summary; status; approved_by?; approved_at?; created_at	meeting,version UQ
meeting_discussion_points	id; meeting_id; summary_id; text; sort_order; evidence_segment_ids uuid[]	summary,sort
meeting_decisions	id; meeting_id; text; confidence?; evidence_segment_ids uuid[]; created_at	meeting
meeting_internal_intelligence	id; meeting_id; ai_run_id; sentiment?; sentiment_reason?; risks jsonb; objections jsonb; opportunities jsonb; recommendations jsonb; created_at	meeting
intelligence_signals	id; organization_id; meeting_id; customer_id?; signal_type; severity; title; description;	org,type,status; customer

	evidence_segment_ids uuid[]; status; created_at	
action_items	id; organization_id; meeting_id; customer_id?; owner_membership_id?; external_owner_name?; description; due_at?; due_date_is_explicit; status; evidence_segment_ids uuid[]; created_at; completed_at?	owner,status,due; customer,status
commitments	id; organization_id; meeting_id; customer_id?; committed_by_type; committed_by_membership_id?; committed_by_name?; description; due_at?; status; evidence_segment_ids uuid[]; timestamps	customer,status; committer
manager_meeting_subscriptions	id; manager_membership_id; subject_type; subject_id?; delivery_mode; include_transcript; include_internal_summary; include_actions; enabled	manager,subject UQ
meeting_emails	id; meeting_id; email_type; status; subject; body_rendered; scheduled_for?; sent_at?; created_by_type; created_at	meeting,status
meeting_email_recipients	id; meeting_email_id; email; display_name?; recipient_type; delivery_status; provider_message_id?; failure_reason?	email_id,email UQ
crm_sync_events	id; meeting_id; customer_id; operation; status; attempt_count; external_reference?; last_error?; created_at; completed_at?	meeting,operation; status
notifications	id; organization_id; recipient_membership_id; type; title; body; entity_type; entity_id; read_at?; created_at	recipient,read_at,created
audit_events	id; organization_id; actor_type; actor_id?; action; entity_type; entity_id; metadata jsonb; occurred_at	org,occurred; entity

4. RLS matrix

Resource	AM	Manager	Senior Manager	Admin
Own meetings	Yes	Yes	Yes	Org-wide
Direct-report meetings	No	Policy/permission	Policy/permission	Org-wide
Transcript	Own + policy	Direct reports + policy	Policy	Org-wide
Internal intelligence	Own if policy	Direct reports	Policy	Org-wide
People	Own profile	Direct-report directory	Broader directory	Manage
Policies	Effective policy	Effective policy	Effective policy	Manage
Exceptions	Request own	Approve if granted	Approve if granted	Manage
Audit	No	Limited if granted	Limited if granted	Read

5. Storage

```
Private Supabase Storage

meeting-recordings/{organization_id}/{meeting_id}/{recording_id}/...
meeting-exports/{organization_id}/{meeting_id}/...

No public transcript/recording buckets.
Use authorized signed URLs or server routes.
```

6. Critical constraints

- Unique canonical meeting per organization/provider/external_event_id.
- Unique bot generation per meeting.

- Unique transcript version per meeting.
- Unique recipient per meeting email.
- One default policy set per organization.
- Manager relationship cannot cross organizations.
- External-summary APIs/renderers cannot automatically join internal-intelligence data.
- Customer, meeting, owner and hierarchy relationships must remain organization-consistent.

7. Domain events

Event	Action
calendar_event.changed	Refresh canonical meeting; enqueue policy
meeting.policy_record	Ensure bot intent
meeting.policy_exclude	Cancel future bot intent if safe
bot.completed	Enqueue transcript
transcript.completed	Enqueue AI
ai.external_summary.completed	Evaluate email policy
ai.internal.completed	Refresh manager intelligence
meeting_email.sent	Audit + CRM activity
action/commitment.changed	Refresh customer memory
provider_subscription.expiring	Renew subscription

06 — Implementation Plan

Goal

Build a production-ready internal pilot in dependency order. Prove Person → Calendar → Policy → Bot → Transcript before heavy investment in AI, email, CRM and advanced intelligence.

Global build constraints

- Provider secrets never reach the browser.
- Vexa-specific concepts never become core domain/UI types.
- No exposed table ships without RLS and allow/deny tests.
- No duplicate bot or duplicate client email is acceptable.
- Internal intelligence cannot enter external email rendering.
- Every async stage has visible status, retry and terminal failure semantics.
- Each milestone must be independently testable.

Milestone	Outcome	Gate
M0	Repository/Foundation	Build, lint, typecheck and tests pass
M1	Organization/Auth/RLS	Cross-org and cross-user deny tests pass
M2	People/Hierarchy	Admin creates AM→Manager relationship
M3	Microsoft	Employee connection + initial sync
M4	Meeting Discovery	Create/update/cancel one canonical meeting
M5	Policy/Exceptions	Deterministic record/exclude decisions
M6	Bot/Vexa	Exactly one bot for one eligible meeting
M7	Transcript	One normalized transcript with speakers
M8	AI	External/internal outputs structurally separate
M9	Meeting Workspace	Useful completed-meeting experience
M10	Client Email	Safe recipients + idempotent send
M11	Manager Intelligence	Direct-report visibility + attention feed
M12	CRM	Eventual idempotent sync
M13	Relationship Memory	Customer timeline + commitments
M14	Ask Signal	Authorized grounded Q&A
M15	Audit/Security	Material actions traceable; leak tests pass
M16	Reliability	Failure drills/reconciliation pass
M17	Internal Pilot	2 → 5 → 10–15 AM rollout

M0 — Repository & Tooling

17. Create monorepo directories for web, domain packages, workers, Supabase and E2E tests.
18. Configure TypeScript strict mode, linting, formatting and test runner.
19. Add environment-variable validation using TRD names.
20. Create CI for typecheck, lint, unit and database tests.
21. Document local setup and secret handling.

M1 — Organization, Auth, Roles & RLS

22. Create organizations, profiles, roles, permissions and memberships migrations.
23. Seed system roles/permissions.
24. Implement server-side membership resolution.
25. Create RLS helper functions and explicit policies.
26. Write allow and deny tests.

27. Create role-aware authenticated app shell.

M2 — People & Hierarchy

28. Create departments/teams.
29. Implement Add Person with work email, role, manager, team and Meeting AI status.
30. Validate same-organization reporting relationships.
31. Build People, Add Person and Person Detail.
32. Audit activation/deactivation/role changes.

M3 — Microsoft Integration

33. Create typed Microsoft provider package.
34. Implement approved connection flow.
35. Persist connection metadata securely.
36. Create/renew calendar subscriptions.
37. Run initial upcoming-event sync.
38. Build integration health state.

M4 — Calendar Discovery

39. Create meeting/attendee/subscription/sync-cursor tables.
40. Implement validated webhook receiver with queue handoff.
41. Implement canonical create/update/cancel.
42. Implement periodic reconciliation.
43. Test duplicate and missed webhook scenarios.
44. Show upcoming meetings from real database state.

M5 — Policy & Exceptions

45. Create policy, decision and exemption tables.
46. Implement pure eligibility function.
47. Add disabled/unsupported/internal/private/sensitive/exemption/external rules.
48. Persist reason code and evaluation snapshot.
49. Build Admin policy UI and AM exception modal.
50. Build approval queue and audit decisions.

M6 — Bot Provider & First Magic Moment

51. Define MeetingBotProvider contract.
52. Implement fake provider for tests.
53. Implement Vexa adapter.
54. Create bot jobs/lifecycle events.
55. Create durable provisioning queue and uniqueness constraint.
56. Normalize provider statuses.
57. Run a real internal Teams meeting test.

FIRST MAGIC MOMENT: Admin adds AM → AM schedules qualifying Teams meeting → Signal detects it → exactly one ApplyWizz Meeting Assistant joins automatically.

M7 — Transcript

58. Create transcript/speaker/segment tables.
59. Fetch provider transcript after completion.

60. Normalize speakers/timestamps.
61. Make ingestion idempotent.
62. Build transcript viewer.
63. Test delayed transcript and retry.

M8 — AI Pipeline

64. Define AIProvider/OpenAI adapter.
65. Create ai_runs and output tables.
66. Extract structured facts, actions and commitments.
67. Generate external-safe summary.
68. Generate internal intelligence separately.
69. Validate schemas and evidence references.
70. Store prompt/model/input versions.

M9 — Meeting Workspace

71. Build meeting header and lifecycle stepper.
72. Build Summary, Transcript and Actions tabs.
73. Build Internal Intelligence for authorized roles.
74. Build Technical Log for Admin.
75. Add processing/failure/reprocess states.

M10 — Client Email

76. Create email/recipient tables.
77. Implement actual-attendee-first recipient resolver with fallback.
78. Create renderer that accepts external-safe objects only.
79. Implement Review Required, Review Window, Immediate and Disabled.
80. Create durable delivery worker and per-recipient status.
81. Add duplicate-send and leakage tests.

M11 — Manager Intelligence

82. Implement reporting authorization.
83. Create manager subscription preferences.
84. Build Manager Overview and My Team.
85. Aggregate risks, overdue commitments, failures and exceptions into Needs Attention.
86. Implement permitted summary/transcript delivery.
87. Verify unrelated-team denial.

M12 — CRM

88. Create CRMProvider and ApplyWizz adapter.
89. Associate meetings with customers.
90. Create sync event/retry worker.
91. Sync meeting, summary, decisions, actions, commitments and next step.
92. Use operation idempotency keys.
93. Surface failures in Admin attention.

M13 — Relationship Memory

94. Build customer meeting timeline.
95. Aggregate open commitments/actions and recent decisions.
96. Aggregate unresolved signals.

97. Generate relationship snapshot.
98. Build Customer Memory screen with source links.

M14 — Ask Signal

99. Implement authorized meeting retrieval.
100. Build Ask This Meeting.
101. Implement authorized customer-history retrieval.
102. Build Ask This Customer.
103. Return source meetings/evidence.
104. Test retrieval isolation.

M15 — Audit & Security

105. Finalize audit taxonomy.
106. Build Audit Log UI.
107. Add RLS deny tests for every exposed table.
108. Add external/internal leakage tests.
109. Add webhook/secret exposure tests.
110. Review retention/deletion behavior.

M16 — Reliability

111. Standardize retry metadata and dead-letter state.
112. Reconcile missing bot/transcript/AI/email/CRM stages.
113. Simulate duplicate/missed calendar events.
114. Simulate Vexa, OpenAI, email and CRM outages.
115. Load-test 15 concurrent meetings and queue backlogs.
116. Create operational runbooks.

M17 — Internal Pilot

117. Pilot Admin + 1 Manager + 2 AMs.
118. Review every captured meeting and recipient outcome.
119. Expand to 5 AMs after reliability gate.
120. Expand to 10–15 AMs after join/transcript/email gates remain stable.
121. Measure detection, join, transcript, AI, leakage, email and CRM metrics.
122. Freeze V1 only after golden scenarios pass.

Testing matrix

Layer	Must cover
Unit	Policy engine, recipient resolver, provider normalization, idempotency, AI schema validation
Database	RLS allow/deny, organization isolation, manager hierarchy, uniqueness
Integration	Microsoft, Vexa, OpenAI, email, CRM adapters
E2E	Enrollment → meeting → bot → transcript → AI → email → CRM
Failure E2E	Missed/duplicate webhook, lobby timeout, provider outage, delayed transcript, downstream failures
Security	Cross-org/team access, external/internal leakage, secrets
Load	15 concurrent meetings, burst calendar events, queue backlog

Production definition of done

- All golden-path scenarios pass.
- Exactly one bot per eligible meeting generation.
- No internal intelligence can be rendered into external email.
- RLS deny tests pass for unrelated users and organizations.
- Missed Microsoft notifications are recovered by reconciliation.
- Bot/transcript/AI/email/CRM failures are visible and retryable.
- External email is idempotent and auditable.
- Admin can explain why a bot joined/did not join and why an email went to each recipient.
- Core Admin, Manager and AM UX is responsive and accessible.
- Internal pilot with 10–15 AMs meets agreed reliability thresholds.

Approval gates

Gate	Required before
PRD	Technical implementation choices
TRD + Backend Schema	Repository scaffolding and migrations
App Flow + UI/UX	Production screen implementation
M0–M6 vertical slice	Downstream AI/email/CRM investment
Internal pilot reliability	Company-wide enablement

Do not start with AI Chat or dashboard polish. First engineering objective: Person → Calendar → Policy → Bot → Transcript.

Appendix — Source-of-Truth Summary

ApplyWizz Signal is a company-controlled meeting intelligence and organizational memory system that automatically captures eligible customer conversations, understands what happened, distributes the right information to the right people, tracks commitments, preserves customer knowledge and gives management visibility without requiring employees to operate a recorder.

User	Experience statement
Account Manager	Schedule your meeting normally. ApplyWizz handles the rest.
Manager	Know what is happening across your team without attending every meeting.
Admin	Control which meetings become company intelligence and ensure automation actually works.
Leadership	Turn customer conversations into searchable organizational memory.

Technical source of truth: ApplyWizz PostgreSQL. Microsoft, Vexa, OpenAI, email and CRM are synchronized providers behind explicit adapters.

Build source of truth: these six documents. If implementation needs to diverge, update the document first rather than silently changing architecture.